

Advanced ASIC Design

Exercise no 3: Fusion Compiler — synthesis and top-down
implementation

Krzysztof Dziembała

2025-06-04

Contents

1	Initial synthesis and optimisation	3
1.1	Adding VDD and VSS	3
1.2	Creating technology library	8
1.3	Alalysis and elaboration	9
1.4	Initial technology mapping	11
1.4.1	Basic parameters after initial mapping	11
1.4.2	Netlist	12
2	Compilation with constraints	13
2.1	Observed layout changes	13
2.2	Report analysis and parameter comparison	16
3	Final scripts and constraints	18
3.1	lab/task1/scripts/task1.tcl	18
3.2	lab/setup/mcmm_setup.tcl	19
3.3	lab/setup/utilities.tcl	20
3.4	src/sdc/dut_toplevel.sdc	20

List of Figures

1.1	Schematic of top level module after elaboration.	9
1.2	Schematic of <code>dut_arbiter</code> after elaboration with visible gates, multiplexers and registers.	10
1.3	Zoomed-in schematic after elaboration, showing mapping to generic technology-independent elements.	10
1.4	Layout and schematic of top level module after initial mapping. . . .	11
1.5	Layout after initial mapping, showing elements from SAEDRVT14 library.	11
2.1	Layout after <code>logic_opto</code>	13
2.2	Layout after <code>initial_place</code>	14
2.3	Layout after <code>initial_opto</code>	14
2.4	Layout after <code>final_place</code>	15
2.5	Layout after <code>final_opto</code> . Changes from <code>final_place</code> are small. . . .	15

1. Initial synthesis and optimisation

1.1 Adding VDD and VSS

VDD and VSS inputs must be added to all modules to serve as power and ground.

```
diff --git a/Lab_3/asic_lab3/src/rtl/sverilog/dut_arbiter.sv
↪ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_arbiter.sv
index 117b916..cebd394 100644
--- a/Lab_3/asic_lab3/src/rtl/sverilog/dut_arbiter.sv
+++ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_arbiter.sv
@@ -66,7 +66,10 @@ module dut_arbiter #(
    output logic[IN_INTERFACE_ID_WIDTH-1:0]   arb_data_source_id,
    output logic                               arb_data_last,
    output logic                               arb_data_valid,
-   input logic                               arb_data_ready
+   input logic                               arb_data_ready,
+
+   input logic VDD,
+   input logic VSS
);

//=====
diff --git a/Lab_3/asic_lab3/src/rtl/sverilog/dut_fifo.sv
↪ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_fifo.sv
index cd0412f..c662848 100644
--- a/Lab_3/asic_lab3/src/rtl/sverilog/dut_fifo.sv
+++ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_fifo.sv
@@ -44,7 +44,10 @@ module dut_fifo #(
    // read interface
    input logic fifo_re,
    output logic[FIFO_WIDTH-1:0] fifo_rdata,
-   output logic fifo_empty
+   output logic fifo_empty,
+
+   input logic VDD,
+   input logic VSS
);

// other local parameters
diff --git a/Lab_3/asic_lab3/src/rtl/sverilog/dut_input_channel_control.sv
↪ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_input_channel_control.sv
index 178df2a..290936e 100644
--- a/Lab_3/asic_lab3/src/rtl/sverilog/dut_input_channel_control.sv
+++ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_input_channel_control.sv
@@ -43,7 +43,10 @@ module dut_input_channel_control #(
    output logic                               in_valid_arb,
    // valid flag
    output logic                               [DATA_WIDTH-1:0] in_data_arb,
    // data
```

```

        output logic                                in_data_last_arb,
    ↪ // indicator of last data in a frame
-   input logic                                arb_in_transferring //
    ↪ indicator of arbitrating of a transfer from the input interface 0
+   input logic                                arb_in_transferring, //
    ↪ indicator of arbitrating of a transfer from the input interface 0
+
+   input logic VDD,
+   input logic VSS
);

//=====
diff --git a/Lab_3/asic_lab3/src/rtl/sverilog/dut_math_wrapper.sv
    ↪ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_math_wrapper.sv
index 9afe0be..8148c73 100644
--- a/Lab_3/asic_lab3/src/rtl/sverilog/dut_math_wrapper.sv
+++ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_math_wrapper.sv
@@ -20,7 +20,10 @@ module dut_math_wrapper #(
    //output interface:
        output logic                                [FIFO_WIDTH-1:0] fifo_data,
        output logic                                fifo_we,
-   input logic                                fifo_full
+   input logic                                fifo_full,
+
+   input logic VDD,
+   input logic VSS
);

@@ -58,7 +61,9 @@ module dut_math_wrapper #(
    dut_multiplier_18x18_comb dut_multiplier_18x18_comb_inst (
        .a(in_a_c),
        .b(in_b_c),
-   .result(prod_c)
+   .result(prod_c),
+   .VDD(VDD),
+   .VSS(VSS)
    );

    // handshake logic
diff --git a/Lab_3/asic_lab3/src/rtl/sverilog/dut_multiplier_18x18_comb.sv
    ↪ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_multiplier_18x18_comb.sv
index 2092724..670b912 100644
--- a/Lab_3/asic_lab3/src/rtl/sverilog/dut_multiplier_18x18_comb.sv
+++ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_multiplier_18x18_comb.sv
@@ -1,7 +1,10 @@
module dut_multiplier_18x18_comb (
    input logic[17:0] a,
    input logic[17:0] b,
-   output logic[35:0] result
+   output logic[35:0] result,
+
+   input logic VDD,
+   input logic VSS
);

//=====

```

```

@@ -32,29 +35,33 @@ module dut_multiplier_18x18_comb (
    dut_multiplier_9x9_comb mult_a0b0 (
        .a(a_0),
        .b(b_0),
-       .product(prd_a0_b0)
-
+       .product(prd_a0_b0),
+       .VDD(VDD),
+       .VSS(VSS)
    );

    dut_multiplier_9x9_comb mult_a1b0 (
        .a(a_1),
        .b(b_0),
-       .product(prd_a1_b0)
-
+       .product(prd_a1_b0),
+       .VDD(VDD),
+       .VSS(VSS)
    );

    dut_multiplier_9x9_comb mult_a0b1 (
        .a(a_0),
        .b(b_1),
-       .product(prd_a0_b1)
-
+       .product(prd_a0_b1),
+       .VDD(VDD),
+       .VSS(VSS)
    );

    dut_multiplier_9x9_comb mult_a1b1 (
        .a(a_1),
        .b(b_1),
-       .product(prd_a1_b1)
-
+       .product(prd_a1_b1),
+       .VDD(VDD),
+       .VSS(VSS)
    );

    always_comb sum_of_prd = {{18{1'b0}},prd_a0_b0} +
diff --git a/Lab_3/asic_lab3/src/rtl/sverilog/dut_multiplier_9x9_comb.sv
↪ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_multiplier_9x9_comb.sv
index 5f0ad6e..c6256cb 100644
--- a/Lab_3/asic_lab3/src/rtl/sverilog/dut_multiplier_9x9_comb.sv
+++ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_multiplier_9x9_comb.sv
@@ -1,7 +1,10 @@
module dut_multiplier_9x9_comb (
    input logic [8:0] a,
    input logic [8:0] b,
-   output logic [17:0] product
+   output logic [17:0] product,
+
+   input logic VDD,
+   input logic VSS
);

```

```

always_comb product = a*b;
diff --git a/Lab_3/asic_lab3/src/rtl/sverilog/dut_output_control.sv
↪ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_output_control.sv
index ba84824..5a39e59 100644
--- a/Lab_3/asic_lab3/src/rtl/sverilog/dut_output_control.sv
+++ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_output_control.sv
@@ -47,7 +47,10 @@ module dut_output_control #(
    output logic [DATA_WIDTH-1:0] out_data,
    output logic [IN_INTERFACE_ID_WIDTH-1:0] out_data_source_id,
    output logic out_data_last,
- output logic out_last_data_sent //
↪ indicator that last output data has been sent out in a given frame - register
+ output logic out_last_data_sent, //
↪ indicator that last output data has been sent out in a given frame - register
+
+ input logic VDD,
+ input logic VSS
);

//=====
diff --git a/Lab_3/asic_lab3/src/rtl/sverilog/dut_processing_control.sv
↪ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_processing_control.sv
index 92f88c1..1f38af5 100644
--- a/Lab_3/asic_lab3/src/rtl/sverilog/dut_processing_control.sv
+++ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_processing_control.sv
@@ -56,7 +56,10 @@ module dut_processing_control #(
    output logic in2_arb_mode_id_en,

    // first cycle indicator
- output logic first_cycle_of_proc_req
+ output logic first_cycle_of_proc_req,
+
+ input logic VDD,
+ input logic VSS
);

//=====
diff --git a/Lab_3/asic_lab3/src/rtl/sverilog/dut_toplevel.sv
↪ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_toplevel.sv
index 794a020..f73b124 100644
--- a/Lab_3/asic_lab3/src/rtl/sverilog/dut_toplevel.sv
+++ b/Lab_3/asic_lab3/src/rtl/sverilog/dut_toplevel.sv
@@ -65,7 +65,10 @@ module dut_toplevel #(
    input logic out_ready,
↪ // ready flag
    output logic [DATA_WIDTH-1:0] out_data,
↪ // data
    output logic [IN_INTERFACE_ID_WIDTH-1:0] out_data_source_id,
↪ // source ID (indicator of an input interface from which the data is taken)
- output logic out_data_last
↪ // indicator of last data in a frame
+ output logic out_data_last,
↪ // indicator of last data in a frame
+
+ input logic VDD,
+ input logic VSS

```

```

);

//=====
@@ -183,7 +186,10 @@ module dut_toplevel #(
    .in1_arb_mode_id_en      (in1_arb_mode_id_en_c),
    .in2_arb_mode_id_en      (in2_arb_mode_id_en_c),

-   .first_cycle_of_proc_req (first_cycle_of_proc_req_c)
+   .first_cycle_of_proc_req (first_cycle_of_proc_req_c),
+
+   .VDD(VDD),
+   .VSS(VSS)

);

@@ -209,7 +215,10 @@ module dut_toplevel #(
    .in_valid_arb      (in0_valid_c),
    .in_data_arb        (in0_data_c),
    .in_data_last_arb   (in0_data_last_c),
-   .arb_in_transferring (arb_in0_transferring_c)
+   .arb_in_transferring (arb_in0_transferring_c),
+
+   .VDD(VDD),
+   .VSS(VSS)
);

//=====
@@ -234,7 +243,10 @@ module dut_toplevel #(
    .in_valid_arb      (in1_valid_c),
    .in_data_arb        (in1_data_c),
    .in_data_last_arb   (in1_data_last_c),
-   .arb_in_transferring (arb_in1_transferring_c)
+   .arb_in_transferring (arb_in1_transferring_c),
+
+   .VDD(VDD),
+   .VSS(VSS)
);

//=====
@@ -259,7 +271,10 @@ module dut_toplevel #(
    .in_valid_arb      (in2_valid_c),
    .in_data_arb        (in2_data_c),
    .in_data_last_arb   (in2_data_last_c),
-   .arb_in_transferring (arb_in2_transferring_c)
+   .arb_in_transferring (arb_in2_transferring_c),
+
+   .VDD(VDD),
+   .VSS(VSS)
);

//=====
@@ -307,7 +322,10 @@ module dut_toplevel #(
    .arb_data_source_id (arb_data_source_id_c),
    .arb_data_last       (arb_data_last_c),
    .arb_data_valid      (arb_data_valid_c),
-   .arb_data_ready      (arb_data_ready_c)
+   .arb_data_ready      (arb_data_ready_c),

```



```

+
+   .VDD(VDD),
+   .VSS(VSS)
+ );

//=====
@@ -331,7 +349,10 @@ module dut_toplevel #(
    //output interface:
    .fifo_data                (fifo_wdata_c),
    .fifo_we                  (fifo_we_c),
-   .fifo_full                (fifo_full_c)
+   .fifo_full                (fifo_full_c),
+
+   .VDD(VDD),
+   .VSS(VSS)
+ );

//=====
@@ -352,7 +373,10 @@ module dut_toplevel #(

    .fifo_re                  (fifo_re_c),
    .fifo_rdata                (fifo_rdata_packed_c),
-   .fifo_empty                (fifo_empty_c)
+   .fifo_empty                (fifo_empty_c),
+
+   .VDD(VDD),
+   .VSS(VSS)
+ );

@@ -377,7 +401,10 @@ module dut_toplevel #(
    .out_data                  (out_data_c),
    .out_data_source_id        (out_data_source_id_c),
    .out_data_last              (out_data_last_c),
-   .out_last_data_sent        (out_last_data_sent_c) //
+   ↪ indicator that last output data has been sent out in a given frame - register
+   .out_last_data_sent        (out_last_data_sent_c), //
+   ↪ indicator that last output data has been sent out in a given frame - register
+
+   .VDD(VDD),
+   .VSS(VSS)
+ );

//=====

```

1.2 Creating technology library

Running `create_lib`, followed by `report_ref_libs` produces the following output:

```

fc_shell> create_lib ${ResultsDir}/${DesignLibrary} -technology $TechFile
↪ -ref_libs ${RefLib}
Information: Loading technology file
↪ '/eda/synopsys/files/FC_Labs/common/tf/saed14nm_1p9m.tf' (FILE-007)
{dut_toplevel.dlib}
fc_shell> report_ref_libs

```

```

*****
Report : Reference Library Report
Library: dut_toplevel.dlib
Version: V-2023.12
Date   : Wed May 28 13:37:30 2025
*****

```

```

      Name          Path
↳ Location

↳ -----
** saed14rvt_frame_timing
↳ /eda/synopsys/files/FC_Labs/common/ndm/saed14rvt_frame_timing.ndm
↳ /eda/synopsys/files/FC_Labs/common/ndm/saed14rvt_frame_timing.ndm
"*" = Library currently open
"+" = Library has technology information
1

```

1.3 Alalysis and elaboration

For analysis and elaboration to succeeded, `dut_params_pkg.sv` must be removed, because it conflicts with `dut_params_pkg.svh`.

Elaborated project has hierarchy (Fig. 1.1) and is represented using technology-independent elements (Fig. 1.2) from `WVGTECH` library (Fig. 1.3). At this stage, layout does not contain anything useful.

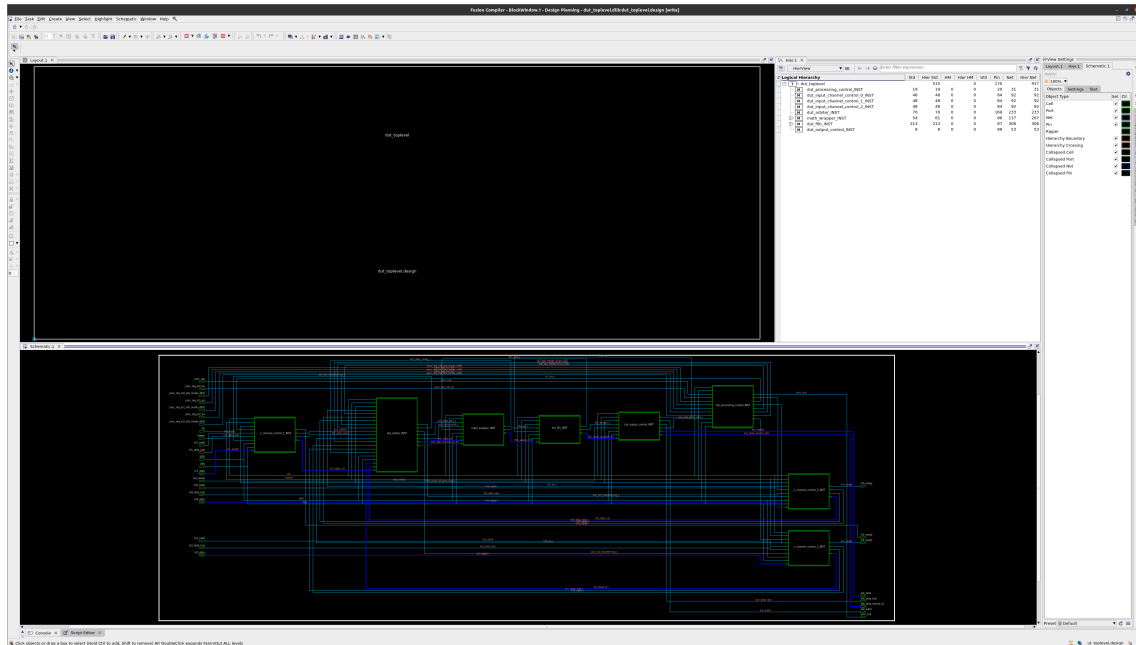
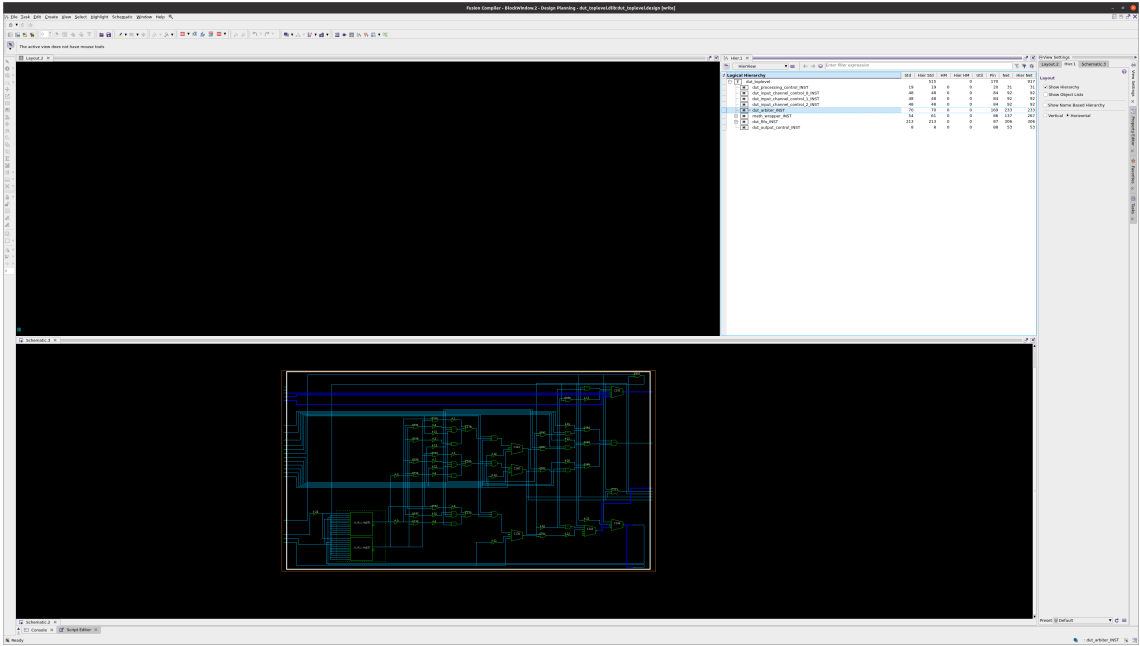


Figure 1.1: Schematic of top level module after elaboration.



1.4 Initial technology mapping

After the `initial_map` step of compilation, the hierarchy is flattened (Fig. 1.4), layout is populated with lots of overlapping cells (Fig. 1.4) and elements in layout use elements available in the provided technology library (Fig. 1.5).

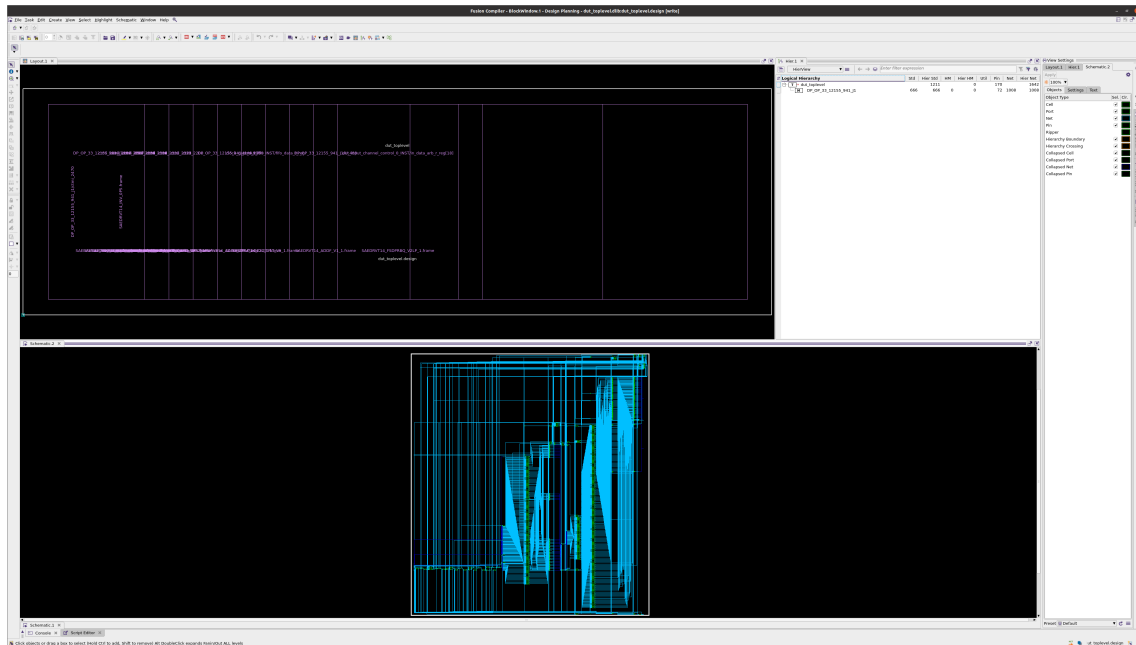


Figure 1.4: Layout and schematic of top level module after initial mapping.

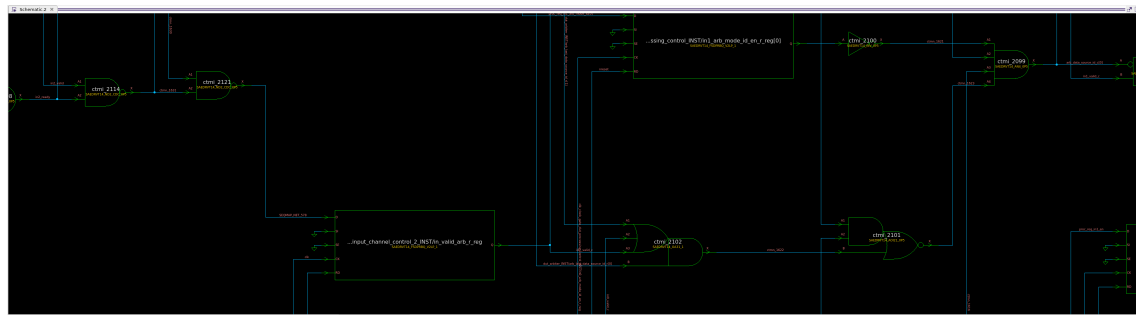


Figure 1.5: Layout after initial mapping, showing elements from SAEDRVT14 library.

1.4.1 Basic parameters after initial mapping

Because there is no constraint information at this point, reports generated at this point are not useful. `report_timing`, for example, prints only

No paths.

Power report produces output, but makes assumptions about voltage (0.7 V), temperature (125 °C) and clock (1 GHz). Reported total power is 3.57×10^7 pW, including:

- Total dynamic power: 1.23×10^7 pW

- Cell internal power: 9.17×10^6 pW
- Net switching power: 3.12×10^6 pW
- Cell leakage power: 2.34×10^7 pW

Power distribution between sequential and combinational parts of the circuit is almost 50/50 (50.4% sequential, 49.6% combinational). Sequential group has higher internal power, while combinational uses more switching and leakage power.

Reported cell area is 906.51, 475.39 is combinational area, 431.12 noncombinational and remaining 10.30 is used by buffers and inverters. The circuit has 1211 cells (872 combinational and 339 sequential), 1714 nets and 58 buffers/inverters.

1.4.2 Netlist

The generated netlist contains modules `dut_toplevel` and `DP_OP_33_12155_941_J3_H2_D0`, which. with the exception of `dut_toplevel` instantiating `DP_OP_33_12155_941_J3_H2_D0`, contain a couple of wires and lots of elements from `SADRV14` library, connected using implicit wires.

The fragment of netlist presented below, shows 2 inverters (`SAEDRV14_INV_OP5`), 2 half adders (`SAEDRV14_ADDH_OP5`) and 1 full adder (`SAEDRV14_ADDF_V1_1`), all using library elements:

```

769 SAEDRV14_INV_OP5 ctmi_2451 ( .A ( PI_1[7] ) , .X ( ctmn_1725 ) ) ;
770 SAEDRV14_INV_OP5 ctmi_2468 ( .A ( PI_2[8] ) , .X ( ctmn_1726 ) ) ;
771 SAEDRV14_ADDH_OP5 U_396 ( .A ( N_20 ) , .B ( N_28 ) , .CO ( N_397 ) ,
772     .S ( N_396 ) ) ;
773 SAEDRV14_ADDH_OP5 U_397 ( .A ( N_21 ) , .B ( N_29 ) , .CO ( N_399 ) ,
774     .S ( N_398 ) ) ;
775 SAEDRV14_ADDF_V1_1 U_398 ( .A ( N_37 ) , .B ( N_45 ) , .CI ( N_397 ) ,
776     .CO ( N_401 ) , .S ( N_400 ) ) ;

```

2. Compilation with constraints

Because `mcm_setup.tcl` calls `current_mode` and constraints apply to mode, the `read_sdc` command must be executed after sourcing `mcm_setup.tcl`, to make sure that constraints apply to the *Normal* mode and *Normal_Typical* scenario.

2.1 Observed layout changes

The cell layout changes as follows:

1. After `initial_map` all cells are overlapping like shown previously on Fig. 1.4.
2. `logic_opto` creates rectangular chip area and roughly places cells. Many are overlapping and there are no clear rows. (Fig. 2.1)
3. `initial_place` significantly reorganises cell layout and makes rows slightly more visible. There are still many overlapping cells. (Fig. 2.2)
4. `initial_drc` does not move any of the existing cells, but adds buffer cells (SAEDRVT14_BUF_8).
5. `initial_opto` places the cells in rows and eliminates overlaps. (Fig. 2.3)
6. `final_place` moves lots of cells, but still keeps them in rows and groups of cells visible after `initial_opto` stay in similar places. (Fig. 2.4)
7. `final_opto` moves some cells, but there aren't significant differences from `final_place`. (Fig. 2.5)

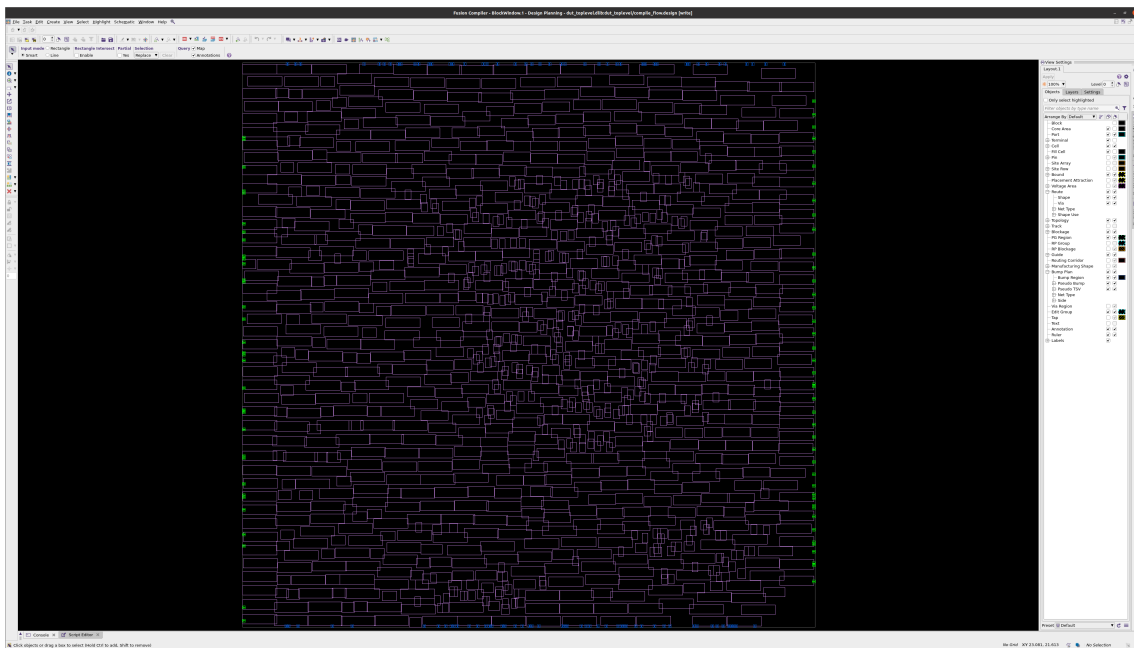


Figure 2.1: Layout after `logic_opto`.

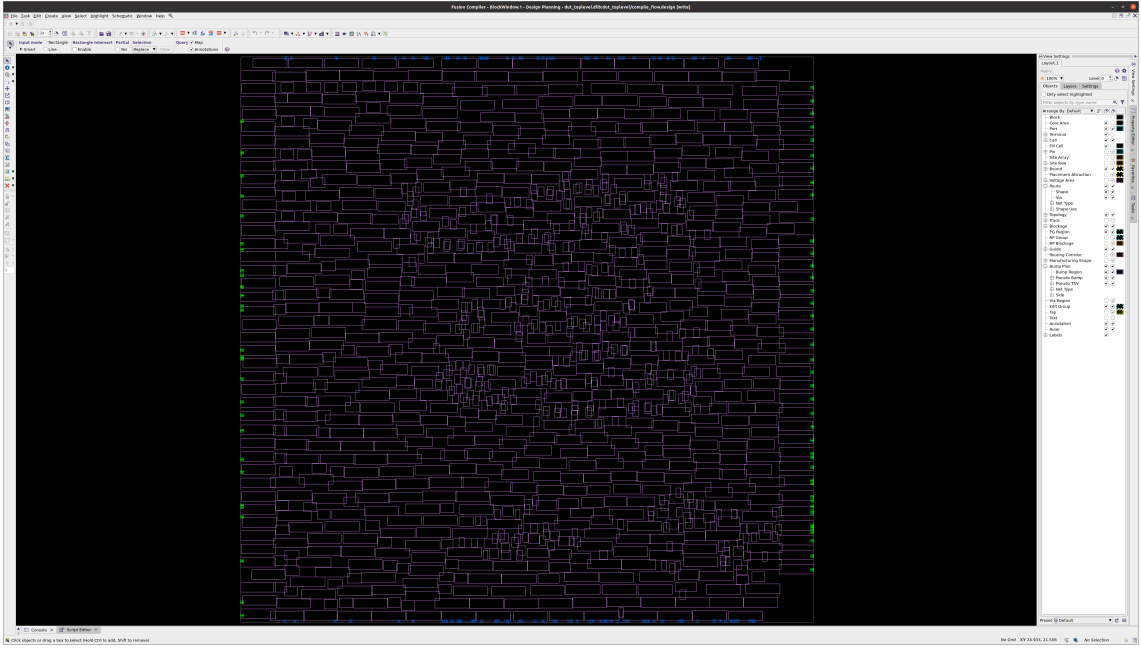


Figure 2.2: Layout after `initial_place`.

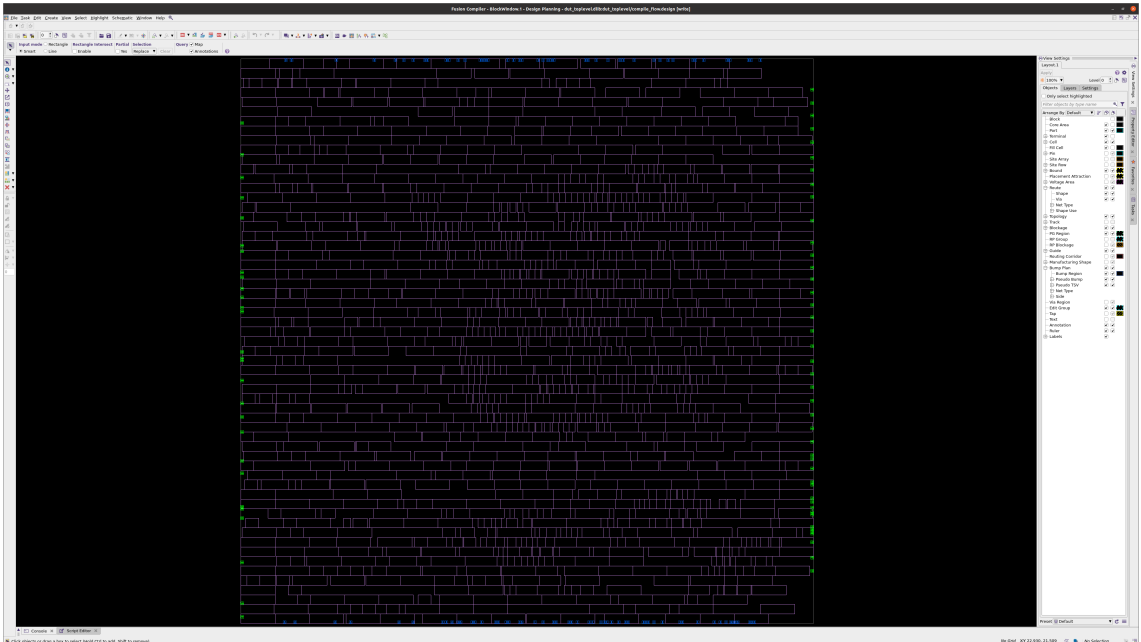


Figure 2.3: Layout after `initial_opto`.

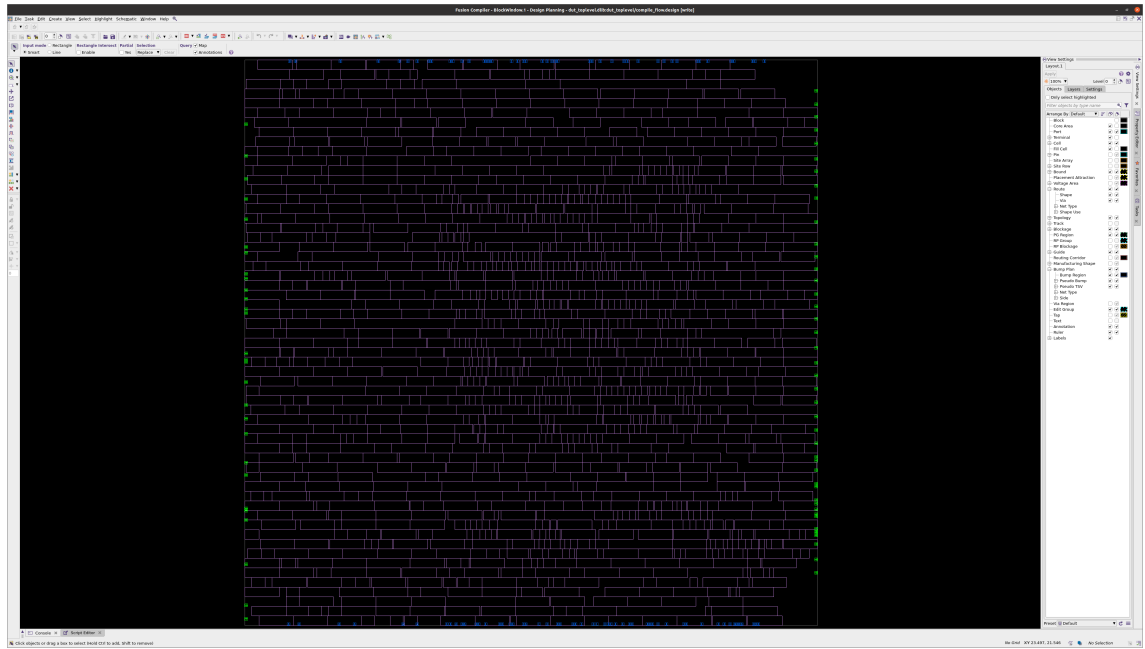


Figure 2.4: Layout after `final_place`.

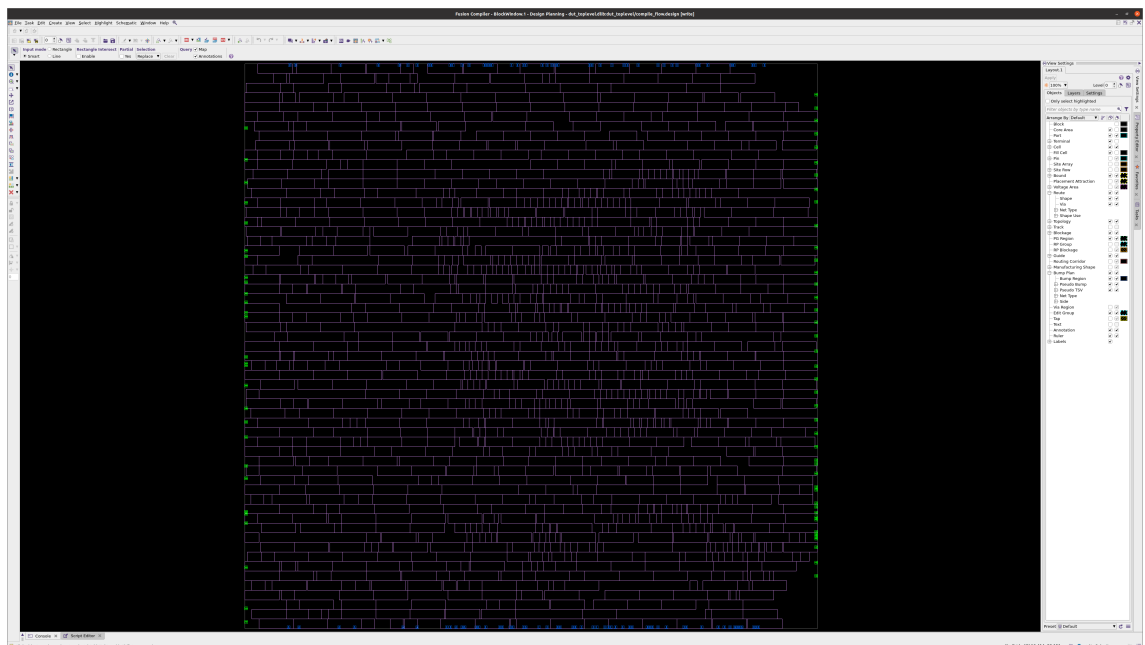


Figure 2.5: Layout after `final_opto`. Changes from `final_place` are small.

2.2 Report analysis and parameter comparison

The Table 2.1 presents a comparison of select parameters from reports after different compilation stages. Analysis of these results shows that `initial_map` has no chip area, as *observed above*, so power and timing details may be very inaccurate.

`initial_drc`, unlike other stages, makes most parameters worse, because it only adds buffers to help with high-fanout nets. All other stages reduce power - both leakage and dynamic, which is 2 orders of magnitude greater. Cell area and cell count are in general reduced as the synthesis progresses, except for `initial_drc` and `initial_opto`, which increases those, while reducing power (especially leakage) and levels of logic.

Timing is improved only in `initial_place` and `final_opto` stages, both times by 0.02ns. All other stages increase the critical path length. While it may seem counterintuitive, the most likely reason is that the constraints are already met, so the compiler can improve other features (power) instead. Constraints specify the clock period as 3ns, but the slack always stays above 1.5ns! This means that the clock could be easily doubled from 333.3MHz to 666.6MHz or even more without resynthesizing the circuit. Given way faster clock, the synthesis stages may behave differently and improve timing, while sacrificing e.g. power.

An interesting parameter reported in QoR is *Nets with violations* in *Design Rules* section (`report_qor -include electrical_drc`), shown in the table as *Nets with DRC violations*. I could not find any details about what exactly these violations are and the documentation does not mention it either.

Legality check done using `check_legality` reports no violations and finishes successfully.

Table 2.1: Comparison of select reported parameters from subsequent compilation stages.

Parameter	initial_map	logic_opto	initial_place	initial_drc	initial_opto	final_place	final_opto
Total power (pW)	3.57×10^7	4.13×10^7	4.09×10^7	4.43×10^7	4.41×10^7	4.39×10^7	4.36×10^7
- Dynamic power (pW)	3.53×10^7	4.10×10^7	4.05×10^7	4.40×10^7	4.38×10^7	4.36×10^7	4.33×10^7
- Leakage power (pW)	3.68×10^5	3.24×10^5	3.24×10^5	3.41×10^5	3.12×10^5	3.12×10^5	3.12×10^5
Critical path length (ns)	0.98	1.30	1.28	1.29	1.36	1.43	1.41
Worst setup slack (ns)	2.01	1.68	1.70	1.69	1.62	1.55	1.57
Worst hold slack (ns)	0.03	0.03	0.03	0.03	0.03	0.03	0.03
Chip area	0.000	1270.506	1270.506	1270.506	1270.506	1270.506	1270.506
Total cell area	906.69	904.25	904.25	915.13	915.97	915.00	914.51
- Combinational area	475.57	473.13	473.13	484.00	484.85	483.87	483.38
- Noncombinational area	431.12	431.12	431.12	431.12	431.12	431.12	431.12
Number of nets	1716	1608	1608	1621	1646	1646	1645
Number of cells	1213	1177	1177	1190	1216	1214	1213
- Number of buffers	0	0	0	13	13	13	13
- Number of inverters	59	23	23	23	21	21	21
Levels of logic	48	48	48	48	47	47	47
Nets with DRC violations	38	1	2	1	5	13	11

3. Final scripts and constraints

3.1 lab/task1/scripts/task1.tcl

```
1 source -echo ../../setup/main_setup.tcl
2 source -echo ${SetupDir}/design_setup.tcl
3
4 create_lib ${ResultsDir}/${DesignLibrary} -technology $TechFile -ref_libs
   ↪ ${RefLib}
5 report_ref_libs
6
7 analyze -format sverilog [glob ${SystemVerilogDir}/*.svh,sv]]
8 elaborate ${DesignName}
9 set_top_module ${DesignName}
10
11 save_block -as ${DesignName}/rtl_read
12
13 # start_gui
14
15 compile_fusion -to initial_map
16
17 report_timing
18 report_power
19 report_area
20
21 write_verilog ${ResultsDir}/${DesignName}_initial_synthesis.v
22
23 current_block
24 save_block
25 save_block -as ${DesignName}/initial_synthesis
26
27 get_blocks -all
28 list_blocks
29
30 save_lib
31
32 copy_block -from ${DesignName}/rtl_read -to ${DesignName}/compile_flow
33 open_block ${DesignName}/compile_flow
34
35 source -echo ${SetupDir}/technology_setup.tcl
36 read_sdc -echo ${SdcFile}
37 source -echo ${SetupDir}/mcm_setup.tcl
38 read_sdc -echo ${SdcFile}
39
40 set_lib_cell_purpose -include none {*/*_A021* */*_V2LP*}
41 set_app_options -name place.coarse.continue_on_missing_scandef -value true
42
43 compile_fusion -check_only
44
45 proc compile_and_report {StageName {nofrom "a"}} {
46     if {$nofrom eq "-nofrom"} {
```

```

47         compile_fusion -to $StageName
48     } else {
49         compile_fusion -from $StageName -to $StageName
50     }
51     generateReports $StageName
52 }
53
54 compile_and_report initial_map -nofrom
55 compile_and_report logic_opto
56 compile_and_report initial_place
57 compile_and_report initial_drc
58 compile_and_report initial_opto
59 compile_and_report final_place
60 compile_and_report final_opto
61
62 check_legality
63
64 save_block
65 save_lib
66
67 close_blocks
68 close_lib
69
70 exit

```

3.2 lab/setup/mcmm_setup.tcl

```

1  #####
2  #####                               MCMM Setup
3  ↪ Script                               #####
4  #####
5  # Remove all MCMM related info
6  remove_corners -all
7  remove_modes -all
8  remove_scenarios -all
9
10 # Create Corner(s)
11 create_corner Typical
12
13 # Set parasitics parameters
14 set_parasitics_parameters -early_spec nomTLU -late_spec nomTLU -corners {Typical}
15
16 # Create Mode
17 create_mode Normal
18 current_mode Normal
19
20 # Create Scenario(s)
21 create_scenario -mode Normal -corner Typical -name Normal_Typical
22
23 # Set operating conditions for the corner and scenario
24 current_corner Typical
25 current_scenario Normal_Typical
26 set_operating_conditions ${PVT_TT}
27

```

3.3 lab/setup/utilities.tcl

```
1  proc generateReports {StageName} {
2      variable ReportsDir
3
4      echo "Generating reports for stage: ${StageName}"
5      redirect -file ${ReportsDir}/${StageName}_report_power.rpt {report_power
6      ↪ -scenarios Normal_Typical}
7      redirect -file ${ReportsDir}/${StageName}_report_timing_setup.rpt
8      ↪ {report_timing -scenarios Normal_Typical -delay_type max}
9      redirect -file ${ReportsDir}/${StageName}_report_timing_hold.rpt
10     ↪ {report_timing -scenarios Normal_Typical -delay_type min}
11     redirect -file ${ReportsDir}/${StageName}_report_area.rpt {report_area}
12     redirect -file ${ReportsDir}/${StageName}_report_qor.rpt {report_qor}
13     redirect -file ${ReportsDir}/${StageName}_report_design.rpt {report_design}
14     echo "Generated reports for stage: ${StageName}"
15 }
```

3.4 src/sdc/dut_toplevel.sdc

```
1  set PERIOD 3
2  set CLOCK_NAME clk
3
4  create_clock -period $PERIOD -name $CLOCK_NAME [get_ports $CLOCK_NAME]
```